System Packages & Functions Architecture Guide

DocuConverter Pro Backend Architecture Reference & Package Installation Manual

This document provides a comprehensive technical overview of the Python packages, modules, and helper libraries utilized by each backend conversion and manipulation function within DocuConverter Pro. It also contains detailed step-by-step deployment instructions for setting up these dependencies in production and development environments.

1. External Packages Matrix

The table below lists all external, third-party libraries integrated into the conversion dashboard and utilities, including their import names, verified versions, and primary roles.

Package	Import Name	Version	Primary Role / Functionality
pywin32	win32com / pythoncom	312	Provides native Windows COM integration. Utilized for high-fidelity conversion of Word (docx), Excel (xlsx), and PowerPoint (pptx) using MS Office engines.
PyMuPDF	fitz	1.27.2	High-performance PDF reading, page rendering (pixmaps), annotations (FreeText, Whiteout, Rects), page rotation, and advanced document scrubbing/compression.
python-docx	docx	1.2.0	Programmatically creates, reads, and modifies Microsoft Word XML structures (.docx). Used in PowerPoint outline-to-Word generation and document styling.
openpyxl	openpyxl	3.1.5	Handles Excel workbooks (.xlsx) directly. Powers Excel spreadsheet creation, multi-sheet cell merging, sheet extraction, and splitting operations.
pdfplumber	pdfplumber	0.11.10	Extracts rich layout and tabular structures from PDF pages. Serves as the core engine for converting PDF

			pages back to structured Excel worksheets.
pillow	PIL (Image)	12.2.0	Robust image manipulation library. Handles formats, color modes (RGB/RGBA conversions), page merges, image resizing, and compression algorithms.
mammoth	mammoth	1.12.0	Translates DOCX paragraphs and tables to semantic clean HTML. Utilized during older word-edit pipelines to render previews.
htmldocx	htmldocx	0.0.6	Converts HTML snippets back into structured DOCX content. Utilized for HTML-to-Word saving pipelines.
Django	django	6.0.6	Web server framework supplying the HTTP handlers, routing, templates, file storage interfaces, JSON utilities, and SQLite sessions.

2. Function-by-Function Reference

Below is the granular technical breakdown mapping every backend python function to its dependencies, logical behavior, and target services.

A. Document Editor Services

prepare_editor_html_from_docx(docx_path)

- **Service Location:** pdfapp/services/editor.py
- **Package Dependencies:** pywin32 (win32com.client)
- **Description:** Invokes Microsoft Word via COM (Word.Application) to open the target document and save it in a filtered HTML format. This HTML body is then parsed, cleaned, and returned to the browser to enable the MS-Word style inline document editor.

html_to_docx_preserving_layout(html_content, output_path, base_docx_path)

- **Service Location:** pdfapp/services/editor.py
- **Package Dependencies:** pywin32 (win32com.client), built-in HTMLParser, difflib.SequenceMatcher
- **Description:** Examines edits submitted from the web interface. It parses the body text blocks from HTML, compares them with the original paragraphs in the base Word file using SequenceMatcher, and applies text replacements in-place to a copy of the original document. This ensures formatting, tables, page margins, and fonts are preserved with 100% fidelity without structural deformation.

B. Core Conversion Services

safe_docx_to_pdf(docx_path, pdf_path)

- **Service Location:** pdfapp/services/word_to_pdf.py
- **Package Dependencies:** pywin32 (win32com.client, pythoncom)
- **Description:** Initializes COM and uses Microsoft Word to open the DOC/DOCX document, exporting it natively as a PDF (using WdExportFormat.wdExportFormatPDF). This offers the highest possible rendering quality, preserving headers, footers, charts, and embedded fonts.

safe_pdf_to_word(pdf_path, docx_path)

- **Service Location:** pdfapp/services/pdf_to_word.py
- **Package Dependencies:** pywin32 (win32com.client, pythoncom)
- **Description:** Leverages Microsoft Word's native PDF Reflow engine through COM. It opens a PDF file within Microsoft Word, converts it internally into an editable Word document flow, and saves it as DOCX. This preserves paragraphs, layouts, and tables far better than external command-line libraries.

safe_excel_to_pdf(excel_path, pdf_path)

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** pywin32 (win32com.client)
- **Description:** Controls Microsoft Excel via COM to open the workbook and export all worksheets as a single PDF. Excel auto-calculates pages and fits tabular columns natively before rendering.

safe_powerpoint_to_pdf(ppt_path, pdf_path)

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** pywin32 (win32com.client)
- **Description:** Automates Microsoft PowerPoint using COM. It opens the presentation (.ppt/.pptx) and exports slides into PDF format with exact layout, color profiles, gradients, and font matching.

safe_pdf_to_excel(pdf_path, excel_path)

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** pdfplumber, openpyxl
- **Description:** Extracts structured tables from PDF pages. If a table structure is identified, it extracts row and cell values and inserts them into sheet cells. If no table elements are found on a page, it falls back to raw text extraction, writing lines into the worksheet sequentially.

safe_image_to_pdf(image_paths, pdf_path)

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** pillow (PIL.Image)
- **Description:** Loads a list of physical image files (JPG, PNG, JPEG), converts them to standard RGB color models, and compiles them together into a single multi-page PDF document.

C. PDF Editing & Operations

safe_edit_pdf(input_path, output_path, actions_json)

- **Service Location:** pdfapp/services/pdf_service.py
- **Package Dependencies:** PyMuPDF (fitz), json
- **Description:** Applies direct editing commands to the PDF structure without DOCX conversion. It processes actions: (1) Adds annotation objects (FreeText) for new text, (2) Draws white rectangles for whiteouts, (3) Draws rect shapes with opacity for highlights, (4) Appends vector line paths for freehand drawings, (5) Modifies page rotation angles (90, 180, 270 degrees).

D. Document Compression Services

compress_pdf(input_path, output_path)

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** PyMuPDF (fitz)
- **Description:** Reduces PDF size by scrubbing unnecessary metadata, XML streams, attached files, embedded thumbnails, and saving it with object stream compression (deflating fonts and downscaling image resolutions).

compress_docx(input_path, output_path) / compress_powerpoint / compress_excel

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** zipfile, pillow (PIL.Image), io
- **Description:** Opens Office OpenXML archives (.docx, .pptx, .xlsx) as zip packages, locates internal media directories, extracts raster images, resizes them if they exceed the max width, recompresses them to optimized JPEGs or PNGs, and rebuilds the archive structure.

E. Excel Manipulation Services

merge_excel_files(file_paths, output_path)

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** openpyxl
- **Description:** Loads multiple workbook files, reads all active sheets in data-only mode, and creates worksheets in a master workbook, renaming sheets with safe unique identifiers.

split_excel_file(file_path, page_range_str, fs)

- **Service Location:** pdfapp/views_legacy.py
- **Package Dependencies:** openpyxl
- **Description:** Reads sheets from an Excel workbook, extracts requested indexes or names, copies cell values, and saves a separate Excel file for each sheet.

3. Installation & Configuration Manual

A. Development & Production Package Installation

To install all required libraries in your Python environment or virtual environment, execute the following command:

```
pip install Django==6.0.6 pywin32==312 PyMuPDF==1.27.2 python-docx==1.2.0
openpyxl==3.1.5 pdfplumber==0.11.10 pillow==12.2.0 mammoth==1.12.0
htmldocx==0.0.6
```

B. Windows COM Automation Requirements

Because Microsoft Word, Excel, and PowerPoint COM interfaces are used for high-fidelity conversion, the target server or system must satisfy the following system constraints:

- **Operating System:** Microsoft Windows is strictly required. Non-Windows environments (Linux, macOS) do not support the Win32 COM library or Microsoft Office application services.
- **Microsoft Office Installation:** Microsoft Office (containing Word, Excel, and PowerPoint) must be installed locally on the system, and activated with a valid license.
- **Post-Installation Script (pywin32):** After installing the pywin32 library, you must run the post-installation script to register the COM dlls and copy required dependencies to the system path. Execute:

  ```
  python Scripts/pywin32_postinstall.py -install
  ```

C. IIS Web Server Deployment Configuration

If deploying the Django application behind a Windows service or IIS (Internet Information Services), the system user executing the application pool must have interactive access permissions for COM objects. Follow these steps:

- **Configure Application Pool Identity:** Ensure that the IIS Application Pool running your web service is configured to run as a custom local administrator account (or "LocalSystem" for development testing). The default "IIS_IUSRS" or "ApplicationPoolIdentity" does not have permission to launch desktop applications like Word.
- **Create Desktop Folders (Critical):** Microsoft Office applications (Word/Excel) running in non-interactive session modes require specific physical folders to store temporary environment structures. If these folders are missing, COM initialization will throw access denied errors. Create the following empty directories manually:

  ```
  C:\Windows\System32\config\systemprofile\Desktop
  ```

```
C:\Windows\SysWOW64\config\systemprofile\Desktop
```

Once these configurations are applied, restart the Django application. The application will automatically detect the presence of the Microsoft Office backend and handle all conversions seamlessly.